The BRITISH UNIVERSITY IN EGYPT

AI Planning for Robot Systems

Final Report



Group Members

Name	Student ID
Jana Mostafa	232789
Seif Emad	236467
Seif Mahmoud	232470
Omar Nassar	233042

Semester One, 2024–2025

Contents

1 Introduction **1**

2 System Pipeline Overview **3**

3 Design Approach **5**
3.1 Algorithms 5
3.2 Navigation and Obstacle Avoidance 7

4 Implementation Details **9**

5 Testing and Results **13**
5.1 Performance 15

6 Conclusion **17**

List of Figures

2.1 End-to-end system pipeline. 4

4.1 System enviroment with obstacles 9

5.1 Performance results (e.g., comparison between different path planning algorithms 14
5.2 Performance results (e.g., Line plot comparison between 2 algorithms . . 14
5.3 Performance results (e.g., Bar plot comparison between 2 algorithms . . . 15

List of Tables

5.1 Comparative Analysis of Q-Learning vs. Swarm Heuristic Approach . . . 16

Chapter 1

Introduction

This work focuses on the design and implementation of the autonomous planning system for a **home cleaning robot** inside a structured, dynamic indoor environment. Using the `Pygame` framework and based on basic principles of **ROS/SLAM**, our focus will be on developing a **Hybrid Path Planner**. The system will make use of **A*** or **Dijkstra's Algorithm** in creating the most optimized global path around static house obstacles and will be combined with a reactive method to tackle the high-stakes challenge of **real-time avoidance** of unpredictable dynamic obstacles, such as moving humans and pets. The first major deliverable is the successful simulation of this robust navigation strategy that demonstrates core competencies in mobile robotics autonomy.

The current document outlines the deployment and measurement of two state-of-the-art robotic coordination techniques for autonomous house cleaning, which are multi-agent swarm intelligence and reinforcement learning through Q-learning. The system overcome the problem of effective dirt collection in a grid-based area by parallel cooperative cleaning through swarm robotics and sequential adaptive learning through Q-learning with hyper-parameter tuning via Particle Swarm Optimization (PSO). The main goal is to showcase the teamwork of several autonomous agents in cleaning a habitat very effectively, at the same time, investigating ways of optimizing reinforcement learning to enhance the performance of single-agent sequential cleaning.

- Project objectives are
    1. Design & Implement a Hybrid Path Planner
    2. Achieve Autonomous Mapping & Localization
    3. Validate Coverage and Efficiency

Sensors:

1. Lidar/Range Sensor is used in the (`ekf_update`) function for detecting static and dynamic obstacles.
2. Dirt sensors are used by checking `grid[robot.row][robot.col].has_dirt` then directly calling the `remove_dirt()` function, which guarantees that the robot senses its cleaning target.
3. An obstacle sensor (static/dynamic) is used by checking `grid[nr][nc].is_barrier` and `getattr(grid[nr][nc], 'has_dynamic', False)` before moving to a cell,

which provides the needed collision information for path planning around static and dynamic obstacles.

Actuators:

1. Drive Motors/Wheels: The Robot uses `self.speed` and the `move_towards` method to change the robot's coordinates (`self.x` and `self.y`), consequently executing planned motion.
2. Cleaning Mechanism: `grid[robot.row][robot.col].remove_dirt()` is executed when the robot reaches a dirty cell, which represents actuators like a vacuum or mop performing the home cleaning function.

Project Scenario: Autonomous Home Cleaning

- **Scenario:** Our project simulates an autonomous mobile robot tasked with **cleaning a house environment**.
  - **Environment:** A structured indoor space (house or floor plan) represented by a **2D Occupancy Grid Map**.
  - **Static Elements:** Fixed obstacles such as **walls, tables, and furniture**
  - **Dynamic Elements: Unpredictable moving obstacles** such as a cat or human
  - **Task:** The robot must achieve **optimal area coverage** to locate and eliminate all "dirt spots" placed randomly on the map.

Relevance and challenges faced:

1. **Relevant**

   High Commercial Demand

   Automation of Mundane Tasks

   Structured Environment Application
2. **Challenging**

   Dynamic Obstacle Avoidance

   Hybrid Planning Requirement (Handling both static / dynamic obstacles)

   SLAM Complexity

   Task Optimization

Chapter 2

System Pipeline Overview

The following project will give a high-level overview of our autonomous **Hybrid Navigation Pipeline** and workflow of our robot. Our process starts at the **Perception Layer**, where raw sensor data—Lidar/Odometry—is fed into the **SLAM** module, which is concurrently localizing the robot and generating the dynamic **2D Occupancy Grid Map**. The result of this is the **Global Path**, which is then fed into the stage of **Local Planning**. This stage would be considered a reactive collision methodology (e.g., DWA in future work) and will continuously check for dynamic obstacles (cat, human) and produce instantaneous **Velocity Commands**. This would be an iterative workflow that maintains a short, optimal route but ensures real-time safety during the execution of the task and completion of the cleaning mission.

The robots are cleaning the environment which is a 50×50 grid-based environment, and the system works in a way that it uses three main operational modes. The modes are as follows: Swarm Mode Pipeline:

Initialization of the environment with several robots **(at least 3 agents)** Random placement of dirt **(20 spots plus if there is not enough)** Parallel assignment of targets by using unsupervised dirt detection Planning of the paths with A* algorithm for each robot Simultaneous execution with communication flags that stop the conflict over targets Reassignment until all the dirt is cleared is done continuously

Q-Learning Mode Pipeline

A single robot is initialized at the starting position Finding out the nearest dirt using the breadth-first search 200 episodes of Q-learning training for specific dirt as the goal From the Q-table the policy is extracted The path is followed and dirt is collected The process is repeated until the environment is clean

PSO Hyperparameter Optimization Pipeline

The initialization of the particle swarm (8 particles) is done with random hyperparameter vectors For every particle configuration, the headless Q-learning is executed The length of the path is evaluated as fitness metric The particle's position and velocity are updated based on the best personal and global 6 iterations of the refinement process The best configuration for each optimizer (SGD, Momentum, Adam) is identified

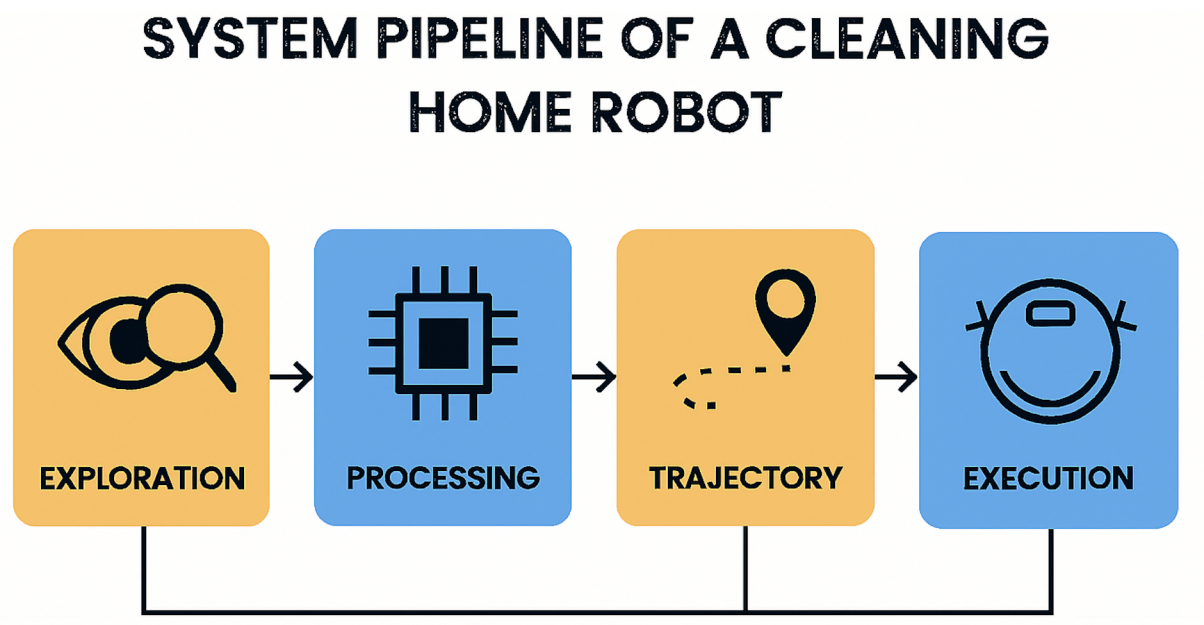


Figure 2.1: End-to-end system pipeline.

Chapter 3

Design Approach

3.1 Algorithms

Mapping and Localization Rationale

For Mapping and Localization, we chose **SLAM** (Simultaneous Localization and Mapping) due to the fact that it solves the challenge of operating in an unknown or dynamic environment by concurrently localizing the robot and generating the **Occupancy Grid Map** while tracking its own position simultaneously. This methodology is crucial for two reasons: firstly, it removes the dependence on a perfectly pre-loaded map, mirroring the autonomy required in real-world scenarios. Secondly, the resulting grid map provides the essential input structure for the subsequent **A*** or **Dijkstra's Algorithm** in the global planning stage.

Global Planning Rationale

* **Rationale behind choosing A*:** A* guarantees that the robot goes through the **optimal shortest path** and achieves superior **computational efficiency** by augmenting the search with a heuristic, typically the Manhattan distance.
* **Rationale behind choosing Dijkstra's Algorithm:** A* is, in theory, Dijkstra's algorithm augmented with a heuristic (an educated guess) to accelerate the search. In using Dijkstra's, you leverage the **gold standard** of shortest-path calculation and thus ensure the **integrity** of your global path's optimality.

Local Planning Rationale

- **Rationale behind choosing DWA:** In each planning cycle, DWA simulates trajectories for different pairs of velocities and scores them on the basis of criteria such as safety with respect to obstacles, goal heading, and velocity. By choosing the highest-scoring trajectory that

avoids predicted collisions, DWA allows the robot to safely and reactively navigate around dynamic obstacles

Swarm Intelligence Algorithm

In a swarm, large numbers of small robots operate primarily based on **independent actions**. However, they communicate information to others using **communication flags**, more precisely by sharing environment information.

The basic components of the algorithm are as follows:

- **Target Selection:** Robots that are not occupied use the function find_nearest_untargeted_dirt(), which performs a **breadth-first search** to discover the closest dirt site not claimed by any other robot, to make full use of it.
- **Reservation System:** The moment a robot selects a target, it assigns grid[target].targeted = True and thus blocks the selection for the other robots.
- **Path Planning:** **A* search** establishes collision-free paths from the robots' locations to their respective targets.
- **Parallel Execution:** All robots clean the dirt when they get there and search for new targets instantly while going on their paths.

This technique helps overcome the problem of **redundancy** in the efforts and at the same time ensures that the area is properly covered by the use of implicit coordination through the targeted flag mechanism, which is very efficient.

Q-learning with Reward Shaping

The implementation of reinforcement learning depends on the combination of **tabular Q-learning** and sophisticated **reward shaping**, which is the main reason for the fast convergence.

- **State Space:** Grid positions (r, c) with 4 discrete actions (up, down, left, right).
- **Q-Table:** 3D array with dimensions (rows $\times$ cols $\times$ 4) that keeps the state-action values.
- **Reward Structure:** ... *(Details missing in the original text, add them here)*

The algorithm offers three different optimizer variants:

- **SGD:** Standard Bellman update:

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[R + \gamma \cdot \max_{a'} Q(s',a') - Q(s,a) \right]$$

- **Momentum:** Includes velocity term $v \leftarrow \beta v + \alpha \cdot \text{TD_error}$.
- **Adam:** Adaptive learning with first and second moment estimates.

Particle Swarm Optimization (PSO)

PSO treats the search as an 8-dimensional continuous optimization problem and optimizes seven continuous hyperparameters along with one discrete optimizer choice.

Particle Representation

Each particle is a vector $[\alpha, \gamma, \epsilon_{\text{decay}}, \text{reward_scale}, \text{momentum_}\beta, \text{adam_}\beta_1, \text{adam_}\beta_2, \text{o}$

Update Rules

- **Velocity Update:**

$$v_i \leftarrow \omega \cdot v_i + c_1 r_1 (p_{\text{best}} - x_i) + c_2 r_2 (g_{\text{best}} - x_i)$$

- **Position Update:** $x_i \leftarrow x_i + v_i$ with clamping to parameter bounds.

Fitness and Parameters

- **Fitness Evaluation:** Average path length over 2 repeated headless runs (lower is better).
- **Parameters:** 8 particles, 6 iterations, inertia weight $\omega = 0.5$, cognitive/social coefficients $c_1 = c_2 = 1.0$.

  The optimizer index dimension maps continuous values $[0, 2]$ to discrete choices {sgd, momentum, adam} via rounding.

3.2 Navigation and Obstacle Avoidance

Obstacle Handling Breakdown

- **Static vs. Dynamic Obstacle Handling**
  1. **Static Obstacles Handling** Static obstacles are defined within (`cell.is_barrier`) in our code, which excludes the cells marked as obstacles from the navigable graph during all planning cycles. This is guaranteed by the path planning algorithms we used (**A*** and **Dijkstra's** algorithms) where the calculated path will never enter any cell that we marked as a barrier.
  2. **Dynamic Obstacles Handling** DWA considers dynamic obstacles by continuously computing possible velocity commands for a robot within its dynamic constraints while predicting the motion of the robot and the motion of the surrounding obstacles over a short time horizon. It employs sensor data on moving obstacle detection and, in turn, updates its position in real time. On each planning cycle, DWA simulates trajectories for a set of different pairs of velocities and scores them on the basis of criteria such as safety with respect to obstacles, goal heading, and velocity.
- **Route optimization was achieved in our project by 3 approaches:**

1. **Optimal Shortest Path:** (**A*** and **Dijkstra's**) find the optimal shortest path.
2. **Cost Function (Uniform Cost):** The cost of moving from one cell to another is small and uniform, set at 1 (cost of 1), which models minimal traversal cost across the grid.
3. **Search Pruning:** A* and Dijkstra's use search pruning where **A*** only searches on promising nodes that are closer to the goal (using the heuristic), and **Dijkstra's** skips any previously visited node for which a shorter path has already been found.

article amsmath amssymb booktabs

Swarm Navigation

Swarm Navigation is based on autonomous, decentralized planning:

- **Pathfinding Algorithm:** Every robot runs independent A* pathfinding.
- **Static Obstacle Avoidance:** The A* algorithm inherently avoids static barriers. Is_barrier cells are considered impassable during pathfinding calculation.
- **Heuristic:** Optimal paths are computed based on 4-connectivity grid movement using the **Manhattan distance heuristic** for A* guidance.
- **Dynamic Obstacle Avoidance:** Dynamic avoidance is managed by constantly checking and updating the grid state during movement.
- **Distributed Exploration Map:** The robots update the grid cell position during their move and set the cells to was_visited to visualize a distributed exploration map.

Q-Learning Navigation

Obstacle avoidance is encoded by the learned policy through **negative collision attempt rewards**. During training:

- **Collision Penalty:** Invalid moves, such as trying to move into is_barrier cells or beyond boundaries, receive a large $-\mathbf{10}$ penalty and keep the agent in its current state.
- **Policy Steering:** Hence, the Q-values of collision actions naturally decrease, which in turn steers the policy towards free space.
- **Path Extraction:** Path extraction from the learned Q-table utilizes argmax actions to yield feasible trajectories.

Reward shaping based on distances accelerates learning by providing gradient information towards the goal and reduces the amount of exploration required to find the optimal paths. [Image of Reinforcement Learning Reward Shaping]

Chapter 4

Implementation Details

Simulation Environment and Packages

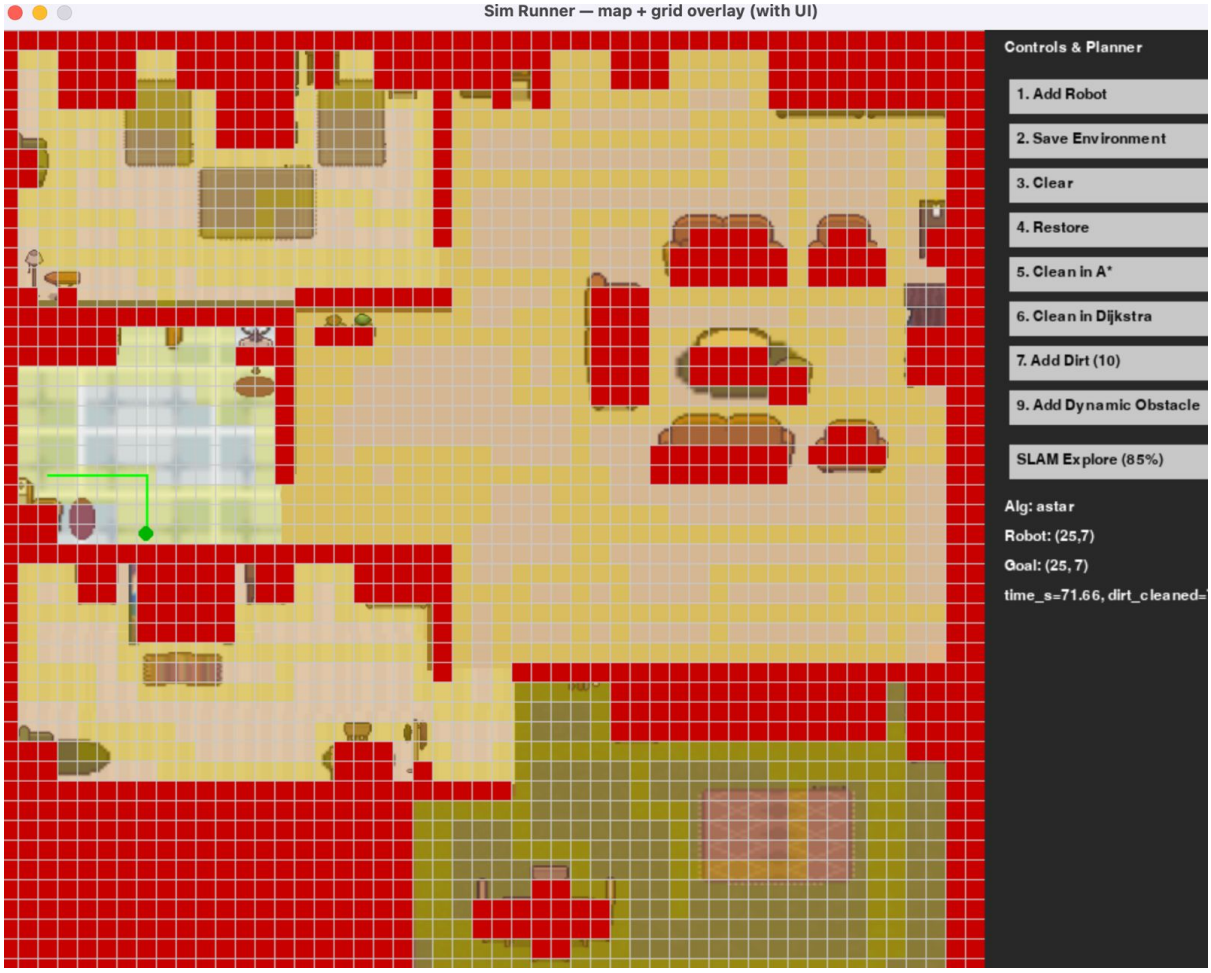


Figure 4.1: System enviroment with obstacles .

- **Simulation environment:** We used **Pygame**, which is a **2D Discrete Grid map simulation** fixed at 50x50 cells (`Rows=50`), with a static background image loaded and overlaid with the discrete planning grid.
- **Packages:**

1. **Numpy:** (array manipulation for occupancy map and EKF/math operations).
2. **Math:** (`math.sin`, `math.cos`, `math.atan2` for motion models).
3. **heapq:** (priority queue for both A* and Dijkstra's search algorithms).

· **Main functions in our system:**

1. **User Interaction** Users interact with the system through a graphical, grid-based interface. The simulator offers options to: [label=→]
2. Draw barriers or walls to show obstacles, such as furniture and room boundaries. (*Function(s):* `draw_barriers()`, `handle_mouse_input()`, `update_grid()`)
3. Add dirt spots to define target areas that the robot must clean. (*Function(s):* `add_dirt()`, `handle_mouse_input()`, `update_grid()`)
4. Place the robot anywhere on the grid to begin the cleaning or exploration process. (*Function(s):* `place_robot()`, `handle_mouse_input()`)
5. Users can create different environment layouts to test performance under various conditions. (*Function(s):* `reset_environment()`, `load_layout()`, `save_layout()`)
6. **Capabilities of the Robot** The robot can perform autonomous exploration, navigation, and cleaning.
   [label=f.]
   (a) **Map Exploration (Simulated SLAM)** The system uses a Simultaneous Localization and Mapping (SLAM) simulation. (*Function(s):* `slam_explore()`, `update_map()`) The robot gradually discovers the environment, mapping and marking visited or unvisited cells. (*Function(s):* `update_coverage_map()`, `mark_cell_visited()`) Exploration continues until approximately 85% of the accessible area is covered. (*Function(s):* `check_exploration_progress()`, `slam_explore()`)
   (b) **Path Planning** The simulator supports **A*** and **Dijkstra's** algorithms. (*Function(s):* `a_star_search()`, `dijkstra_search()`) The robot computes the **optimal path** from its current position to each dirt spot. (*Function(s):* `plan_path_to_target()`, `get_next_dirt_spot()`) The system compares performance between algorithms based on:
   (c) Average cleaning time (in seconds)
   (d) Path length (in grid cells)
   (e) Success rate (percentage of dirt spots cleaned)
   (*Function(s):* `record_metrics()`, `calculate_performance()`, `compare_algorithms()`)
   (f) **Automatic Cleaning** Once mapping is complete, the robot plans and follows paths to each dirt spot automatically. (*Function(s):* `auto_clean()`, `plan_path_to_target()`, `move_robot()`) Cleaning continues until all dirt spots are removed. (*Function(s):* `auto_clean()`, `check_dirt_remaining()`) The sys-

tem ensures efficient coverage while minimizing redundant motion. (*Function(s):* `optimize_path()`, `auto_clean()`)

7. **Graphical User Interface** The system includes a sidebar with interactive control buttons such as:
8. **Add Robot** – places the cleaning robot on the grid. (*Function(s):* `add_robot_button()`, `place_robot()`)
9. **Add Dirt** – adds dirt spots to the environment. (*Function(s):* `add_dirt_button()`, `add_dirt()`)
10. **Clean (A*) / Clean (Dijkstra)** – begins cleaning with the chosen algorithm. (*Function(s):* `start_cleaning_a_star()`, `start_cleaning_dij`
11. **SLAM Explore** – activates autonomous exploration. (*Function(s):* `slam_button()`, `slam_explore()`)

    The GUI visually displays:
12. The robot's movement path (*Function(s):* `draw_path()`, `update_display()`)
13. Cleaned and unexplored areas (*Function(s):* `draw_grid()`, `update_map_displa`
14. Real-time updates during cleaning or exploring (*Function(s):* `update_display()` `refresh_screen()`)
15. **Data Recording and Analysis** The system continuously records key performance indicators, including:
16. Total cleaning time (in seconds)
17. Average path length (in grid cells)
18. Success rate (percentage of dirt spots cleaned)

    (*Function(s):* `record_metrics()`, `log_results()`, `calculate_performance()`
    These metrics are used to compare the performance of **A*** and **Dijkstra's** algorithms. (*Function(s):* `compare_algorithms()`, `export_results()`) Collected data supports evaluation, performance tuning, and optimization analysis. (*Function(s):* `save_results_to_file` `analyze_data()`, `generate_summary()`)

Challenges and Bottlenecks

- **Challenges and Bottlenecks faced:**
- Finding the Shortest Path
- Grid Path Resolution
- Search Speed in Large Maps
- Search Priority Management
- Handling Unpredictable Movement
- Map Representation

Techniques to Combat Challenges

- **Techniques to combat it:**
- **A* algorithm** was used to guarantee an **optimal shortest path**.

- Two algorithms chosen for path planning use **Neighbor-Based Search** to enhance the Grid path resolution.
- **A* combats search speed in large maps** by using the **Manhattan distance as a heuristic** to estimate the straight-line cost to the goal.
- Both **A* and Dijkstra's algorithms** use **priority queues** implemented by (`heapq`) solving the **Search Priority Management** issue.

Swarm

- **Swarm Implementation (Cleaning-Robot.py):**
  1. You can activate run swarm button then this swarm mode can be easily activated using flag draw.swarm active. run_swarm draw.swarm_active.
  2. The find_nearest_untargeted_dirt() function implements the communication mechanism.
- **Q-Learning Implementation (Cleaning-Robot.py):**
  1. The Q-learning mode executes sequentially, training a new policy for each dirt target.
  2. The core training function run_qlearning_demo_blocking() implements the configurable Q-learning algorithm with optimizer support and reward shaping logic.
- **PSO Optimization Implementation (tune_qlearning_pso.py):**
  1. PSO tuner orchestrates per-optimizer optimization campaigns.
  2. The main PSO loop between lines 123 and 165 realizes standard PSO update equations with per-iteration evaluation of particles by subprocess calls to headless mode.
  3. Configuration translation and evaluation are done through environment variable passing as an evaluate_config function.

Chapter 5

Testing and Results

Testing Scenarios

- **Normal operation**
- The robot auto-cleans the grid avoiding all static obstacles in a clear area, and all the static obstacles are already defined within (`is_barrier`).
- **High-traffic conditions**
- The robot calculates the path while multiple dynamic obstacles are moving directly within the same path, blocking the only route to the target dirt spot. This is easily handled in our code by just running the auto-clean feature at high intensity of moving dynamic obstacles.
- **Emergency handling**
- Target cell is blocked by a static barrier. In that case, the robot cleans all the other dirt spots, leaving this exact spot as a permanently blocked spot.
- **Why they represent real world use cases**
  It would not include predefined static obstacles like walls or furniture, which are very common in structured environments like homes or warehouses. It would test basic robot functionality and accuracy in path planning.
  In highly dynamic environments, the robot adapt to dynamic obstacles for example humans or cats reflecting real-life unpredictability, whereby a robot has to keep updating its path in order for it to remain efficient and safe.
  Finally, emergency handling is for rare but critical situations involving permanent obstructions of the target area. In this way, logical decisions like skipping over inaccessible zones are made by the robot without it getting stuck or entering into an infinite loop.
  All these test cases, in their entirety, represent the full range of realistic operating environments and, importantly, demonstrate that the robot can perform with different levels of complexity and uncertainty in a reliable way.

Metric	A*	Dijkstra
Average Cleaning Time (seconds)	**72.87 s**	80.81 s
Average Path Length (cells)	**145 cells**	158 cells
Success Rate (%)	**100%**	100%

Figure 5.1: Performance results (e.g., comparison between different path planning algorithms

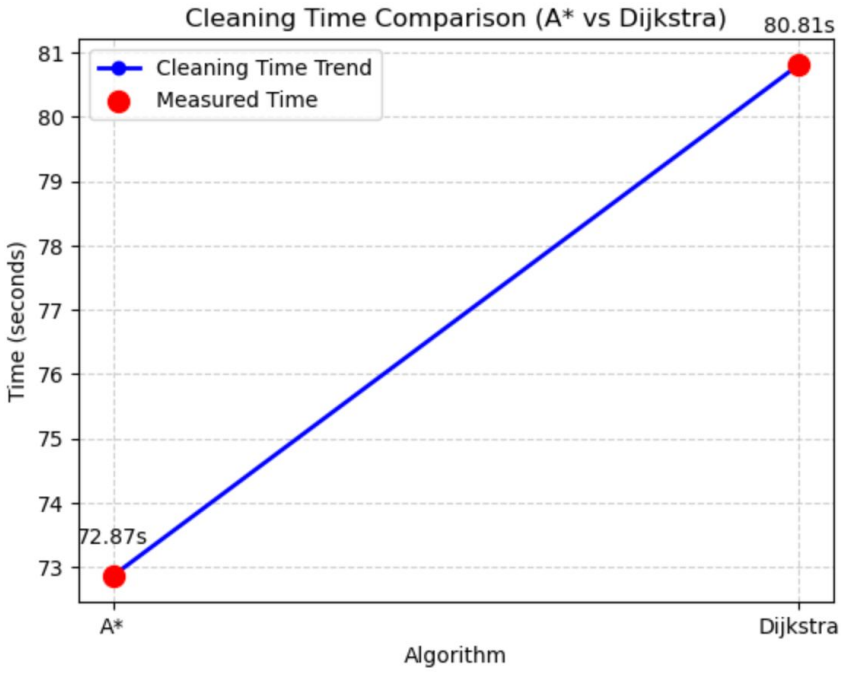


Figure 5.2: Performance results (e.g., Line plot comparison between 2 algorithms .

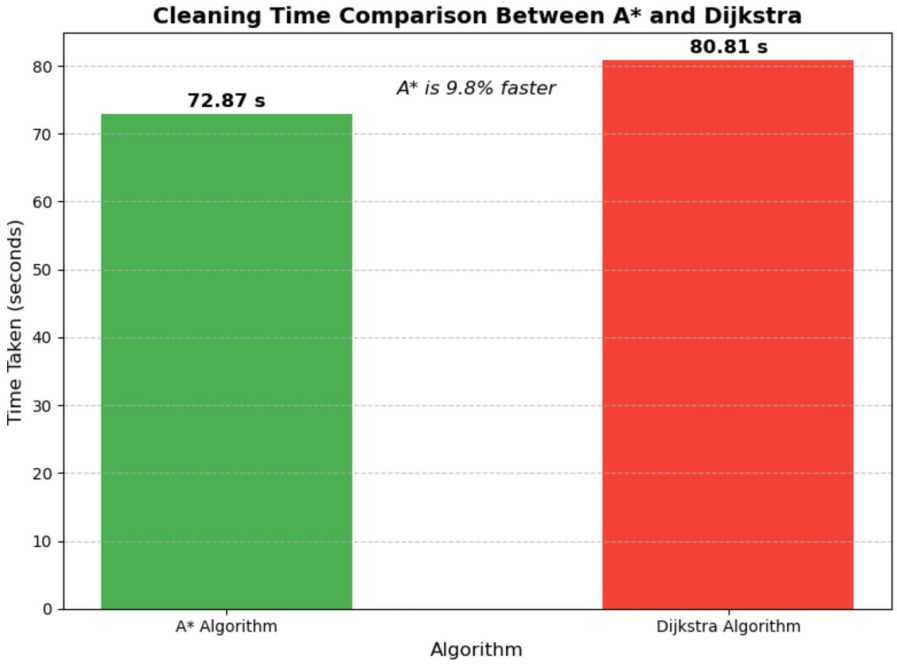


Figure 5.3: Performance results (e.g., Bar plot comparison between 2 algorithms .

5.1 Performance

Q-Learning PSO Optimization Results

The PSO optimization campaign screened a total of **144 configurations** (8 particles $\times$ 6 iterations $\times$ 3 optimizers) and found the optimal configuration for each optimizer type.
The best configuration utilized the **Adam optimizer** and yielded the following hyperparameters:

- Learning rate (α): 0.293
- Discount factor (γ): 0.998
- Epsilon decay: 0.9472
- Reward scale: 0.579
- Adam β_1: 0.913
- Adam β_2: 0.9212

Best Run Performance Metrics

The best configuration obtained, according to the given output file, achieved the following metrics:

- **Path length:** 6 steps. (Very efficient.)
- **Training episodes:** 200
- **Final epsilon:** 0.010 (Successful exploration-exploitation balance.)
- **Convergence:** Epsilon decays rapidly from 1.0 to 0.066 at episode 50.

The system successfully placed 5 dirt spots, and the trained agent found a near-optimal path; thus, reward shaping with PSO-optimized hyperparameters effectively guided learning.

Key Performance Observations

- **Scaling Reward Impact:** The optimised reward_scale of 0.579 (below the default 1.0) indicates that damping the magnitudes of rewards best prevented oscillations in Q-value estimates and allowed stable learning.
- **High Gamma Value:** $\gamma = 0.998$ suggests that the algorithm did benefit from long-term consequences.

Swarm Performance Characteristics

- **Parallelism:** With 3 robots, the theoretical speedup approaches 3× with respect to single-agent sequential cleaning (given no path conflicts).
- **Target Reservation System:** The mechanism of the targeted flag avoids superfluous effort spent by several robots chasing after the same dirt.
- **Scalability:** The decentralized design scales naturally with respect to more robots, mainly limited by physical crowding in densely packed barrier environments.

  The BFS-based nearest untargeted dirt selection guarantees that each robot minimizes its travel distance, while the A* pathfinding ensures collision-free trajectories. article booktabs amsmath

Comparative Analysis

Comparing the two robotic control approaches:

Table 5.1: Comparative Analysis of Q-Learning vs. Swarm Heuristic Approach

Aspect	**Q-Learning**	**Swarm**
Execution Model	Sequential	Parallel
Learning	Adaptive (per-target training)	No learning (fixed heuristic)
Coordination	None (single agent)	Communication flags
Computational Cost	High ($\sim$ 200 episodes $\times$max_steps per target)	Low (direct A* calls)
Optimality	Learned policy from experience	A* optimal paths
Scalability	Fixed single agent	Linear with robot count

Chapter 6

Conclusion

Contributions achieved in our project

- Implementation of Optimal Global Path Planning
- Functional Grid-Based Simulation
- Reactive Dynamic Obstacle Modeling
- Autonomous Mission Logic

Reflect on limitations

- **Static Cost Structure:** The planner cannot distinguish between high-cost areas and low-cost areas.
- **Route Smoothness:** As the A* and Dijkstra's algorithms generate paths consisting of discrete grid-cells, this produces jagged and non-kinematically feasible routes.

Final Conclusion

Summarizing all that was implemented in our final project: this im-Implementation has successfully demonstrated two different schemes of au- Autonomous robotic cleaning, cooperative swarm intelligence and adaptive reinforcement learning. The swarm system enforces efficient multi-agent coordination based on a simple yet effective target-reservation mechanism which allows parallel dirt collection without explicit inter-robot communication protocols. The Q-learning implementation, enhanced with distance-based reward shaping and PSO- Optimized hyperparameters result in near-optimal cleaning policies. converge in 200 training episodes. The automatic optimization of the PSO framework is effective in The tuning of seven hyperparameters across three variants of optimiz- ers, which resulted in an Adam-based configuration with 6-step path lengths during testing.

The PSO optimization framework was found effective for automatic tuning of seven hyperparameters across three variants of optimizers, which resulted in an Adam-based configuration with 6-step path lengths during testing. Primary takeaway points involve the advantage of reward scaling with a reduced magnitude of 0.579, high discounts

of $\gamma = 0.998$ for long-horizon planning, and aggressive epsilon decay at 0.9472 for fast convergence. The adaptive learning rate mechanism of the Adam optimizer outperformed standard SGD and momentum methods.

Future Improvements and work

- Implement a Local Planner (DWA/TEB)
- Introduce Path Smoothing
- ROS/Gazebo Migration

Future work might consider hybrid methods that combine swarm coordination with learned policies, transfer learning to reduce per- target training costs, and dynamic hyperparameter adjustment during execution